Gemini CLI - Conceptual Architecture

February 13, 2026

Authors:

Iram (18ih2@queensu.ca), Lena (22bsn3@queensu.ca), Maciek (20mjb21@queensu.ca), Mack (22NWD3@queensu.ca) Martin (martin.allerdissen@queensu.ca)

Abstract:

This report presents an architectural analysis of Gemini CLI, Google's open-source command-line interface for interacting with the Gemini family of large language models. Gemini CLI is structured around a centralized orchestration model split into two primary subsystems: a terminal-facing interface layer (packages/cli) and a core runtime backend (packages/core). The core subsystem coordinates all communication with the Gemini API, manages tool execution, enforces security policies, and maintains session state, while the CLI layer handles user input, rendering, and interactive workflows.

Our analysis examines the system across five key dimensions: overall functionality, evolution strategy, control and data flow, concurrency model, and the implications for developer responsibility. We find that Gemini CLI follows a turn-based agent loop with a deliberately serialized decision path, where concurrency is limited to overlapped I/O and external process execution rather than parallel reasoning. The architecture supports extensibility through a built-in tool registry and the Model Context Protocol (MCP), which allows external tools to be developed independently. A policy and confirmation layer governs tool execution, ensuring that potentially unsafe operations require explicit user approval. The system evolves through a disciplined release pipeline and a roadmap-driven planning process, with stable extension points that allow new capabilities to be introduced without disrupting the core interaction logic. Overall, Gemini CLI's architecture reflects a platform-oriented design that balances controlled extensibility with predictable, auditable behavior.

Introduction and Overview:

Gemini CLI is an open-source, terminal-based application developed by Google that enables developers to interact with the Gemini large language model directly from the command line. Rather than functioning as a simple chat wrapper, Gemini CLI is designed as an agentic coding assistant—one that can read and write files, execute shell commands, search the web, and invoke external tools on behalf of the user. It is built in TypeScript and organized as a monorepo, with a clear separation between its user-facing terminal interface (packages/cli) and its backend orchestration engine

(packages/core). This architectural split is central to how the system operates, evolves, and distributes development responsibility.

The purpose of this report is to provide a detailed architectural analysis of Gemini CLI, examining both its high-level structure and the design decisions that shape its runtime behavior. Our analysis is organized around five core questions that together offer a comprehensive view of the system.

Section 1 describes what the system does, outlining Gemini CLI's capabilities as an AI-powered terminal assistant and the role each major subsystem plays in delivering that functionality. Section 2 explores how the system evolves, covering its release pipeline (nightly, preview, and stable channels), its roadmap-driven planning process, and the stable extension points such as the tool registry and Model Context Protocol (MCP)—that allow new features to be introduced without destabilizing the core. Section 3 examines the control and data flow among the system's parts, tracing the path from user input through prompt construction, model communication, optional tool execution, and final response rendering. This section highlights how the core subsystem acts as the single point of orchestration, managing all intermediate state and enforcing approval gates before sensitive operations proceed. Section 4 addresses concurrency. While the agent loop itself is deliberately sequential—model step, optional approval gate, tool step, and back to model step—the system does employ asynchronous I/O at its boundaries to remain responsive. This includes non-blocking communication with the Gemini API, concurrent connections to multiple MCP servers, and subprocess-based tool execution. Crucially, none of this concurrency involves parallel decision-making; the orchestrator serializes all reasoning and waits for each tool result before proceeding. Section 5 discusses the implications for how development responsibilities are divided among contributors. The CLI and core split naturally maps to separate teams: interface developers handle command parsing, rendering, and interactive workflows, while core developers manage prompt assembly, model interaction, tool scheduling, and policy enforcement. Tool developers, in turn, can work independently by implementing built-in capabilities or external MCP servers, provided they respect the defined schemas and contracts. Security and governance form their own area of responsibility, centered on the policy and confirmation layer that mediates tool execution.

Across these dimensions, a consistent picture emerges. Gemini CLI's architecture is best characterized as a platform-oriented design centered on coordinated orchestration and controlled extensibility. The core runtime serves as the authoritative coordinator—constructing prompts, interpreting model responses, scheduling tool execution, and enforcing safety policies—while surrounding subsystems handle user interaction, tool implementation, and external integration. This centralized approach simplifies reasoning about system behavior and supports auditability, but it also means that interface stability between subsystems is critical. Changes to event structures,

tool schemas, or policy configurations can have broad effects, and the architecture demands discipline in maintaining compatibility as the system grows.

From an architectural style perspective, Gemini CLI can be understood through the lens of Garlan and Shaw's taxonomy. At its highest level, the system follows a centralized orchestration pattern in which the core subsystem acts as the single control hub, mediating all interactions between the user interface, the language model, and the tool ecosystem. Within this, the agent loop exhibits characteristics of a pipe-and-filter style—user input flows through prompt construction, model inference, tool execution, and response rendering in a well-defined sequence—though the flow is managed by the orchestrator rather than by autonomous pipeline stages. The tool subsystem introduces an implicit invocation style: tools are registered declaratively and invoked dynamically based on model output, meaning the core does not hard-code which tools will run or when. The MCP integration extends this further with a client-server style, where external tool servers communicate over standardized transports and are discovered and invoked at runtime. Finally, the policy and confirmation layer acts as a layered system, sitting between the orchestrator and tool execution to enforce governance rules before actions proceed. These styles coexist within the architecture, each addressing a different concern—orchestration, extensibility, external integration, and safety—and together they support a system that is both modular and predictable.

Architecture:

What does the system do?

Gemini CLI is an AI powered terminal agent that enables developers to accomplish complex tasks through natural language conversation. The system autonomously executes actions on the local environment by invoking tools including file operations, shell commands, web requests, and external service integrations through MCP servers. Unlike simple chatbots that only generate text responses, Gemini CLI acts as an agentic system that reads files, writes code, executes commands, and maintains persistent memory of projects, all while enforcing safety through human approval for destructive operations.

Overall architecture

As shown in Figure 1, the architecture follows a clean separation into four primary layers connected through well defined interfaces. At the presentation layer, the CLI Frontend captures user input from the terminal and displays formatted responses including syntax highlighting, diffs, and streaming updates. This frontend communicates with the Core Runtime through a Core API using an event driven pattern. The Core Runtime serves as the orchestration engine, managing conversation state, constructing prompts with tool definitions and context, streaming requests to the external Gemini API over HTTPS, and coordinating tool execution through the Tool Registry. The Tool Registry acts as a central dispatch hub, maintaining a catalog of available tools and routing invocation requests either to Built-in Tools running within a sandboxed isolation boundary (indicated by the dashed boundary in Figure 1) or to external MCP Servers connected via stdio, HTTP, or SSE

transports. Built-in Tools interact with the Host OS layer through system calls, subprocess execution, filesystem operations, and network requests to perform concrete actions. As illustrated in Figure 1, the system boundary encapsulates the CLI Frontend, Core Runtime, Tool Registry, and Built-in Tools, while the Gemini API and MCP Servers remain external dependencies connected through network protocols, and the Host OS provides the underlying execution environment for all local operations.

How does the system evolve?

Gemini CLI evolves through a mix of a disciplined release process and an architecture that makes it easy to add or change features without breaking the whole system. From the release side, changes don't just appear instantly for everyone. They move through channels (nightly builds, preview releases and then stable) so new features can be tried early, issues can be found and only well-tested changes get promoted to the default version. This is important for architectural evolution, because it allows the team to do internal refactors, like improving how tool execution is coordinated, while still controlling risk and keeping the tool usable for most users.

The project also evolves intentionally through roadmap planning. Instead of being a random collection of features, future work is grouped into areas like tooling, user experience, platform support, security, quality and background/agent features. That gives contributors and maintainers a shared direction and it helps explain why certain changes show up when they do (for example, improvements to safety policies or expansions to integrations and automation).

Most of the long-term evolution is enabled by stable "extension points" in the design. The CLI interface is separated from a reusable core backend, so new frontends (like IDE companions or automated runners) can reuse the same model and tool orchestration logic. The tool system is built to grow over time; new abilities can be added as built-in tools, discovered from external tool servers or packaged and distributed as extensions, all while still fitting into the same execution pipeline. Safety and governance evolve separately through a policy layer, which lets users or administrators adjust what tools are allowed, denied or require confirmation without needing to rewrite tool implementations. Finally, the system is moving toward more advanced agent behavior using subagents and remote agent delegation. This supports scaling from a single assistant into a more specialized or distributed setup while keeping the core interaction loop stable.

What is the control and data flow among parts?

Gemini CLI follows a centralized orchestration architecture in which control and data flow are coordinated through the core subsystem. At a high level, the system operates as an agent loop where user input is collected by the CLI frontend and forwarded to the core backend, which then determines the next action. The core constructs a prompt incorporating the user request and relevant context, sends it to the Gemini API, and receives a model response. Based on this response, control either returns directly to the CLI for rendering or proceeds to tool execution before the final result is returned. This

establishes a predictable control path of the form: CLI → core → Gemini API → (optional) tool execution → core → CLI.

Data flow mirrors this centralized control structure. User input enters through the CLI and is passed as structured data to core, where it is enriched with system state, tool definitions, and prior interaction history. The Gemini API returns structured outputs that may include natural-language responses or tool invocation requests. When tools are invoked, data is exchanged between core and the tool subsystem (built-in tools, MCP servers, or external processes), after which tool outputs are merged back into the interaction context and incorporated into the next model prompt. All intermediate state, including prompts, responses, tool results, and policy decisions, is managed by core, making it the authoritative data coordinator.

The architecture enforces clear control boundaries at approval and execution points. When a tool requires confirmation (e.g., filesystem or shell operations), control flow pauses at the core approval gate until user authorization is received. Only then does data flow proceed to the tool subsystem. This ensures that potentially unsafe actions are explicitly mediated while maintaining a linear decision sequence. Overall, both control and data flow are intentionally centralized and serialized within core, which simplifies reasoning about system behavior, supports auditing and safety enforcement, and enables consistent integration of heterogeneous tools and remote MCP services

What concurrency if any is present?

Gemini CLI is best described as a micromanaged agent loop with a single control center in packages/core and an interactive UI shell in packages/cli. The dominant control structure is not concurrent, but rather a turn-based pipeline: user input is collected in the CLI, forwarded to core, core calls the Gemini API, and then core either returns an answer or executes one (or more) tools before returning a final response for rendering. This interaction sequence is documented as the primary runtime path (CLI -> core -> Gemini API -> tool execution/approval if needed -> final response -> CLI render). Most user-visible work is intentionally serialized to keep the agent's behavior predictable and auditable. In particular, the decision-making loop is sequential: core constructs a prompt, sends it to the Gemini API, receives a response, and only then proceeds. If the model requests a tool that can modify the filesystem or run shell commands, the architecture inserts a synchronization point where execution is paused until the user approves the tool call (i.e., "stop-the-world" from the agent's perspective). Only after approval does core invoke the tool and then fold the tool output back into the next model request. This yields a clear single-threaded conceptual control flow: model step -> (optional) approval gate -> tool step -> model step, rather than speculative parallel actions. Concurrency exists primarily in the form of overlapped I/O and independent external processes, not as multiple simultaneous "agent decisions." Conceptually, concurrency appears at the boundaries between components: (1) the CLI can continue rendering/handling terminal events while core awaits network or tool I/O; (2) tools may run as separate OS processes (e.g., shell execution) and can therefore proceed while the main runtime is awaiting completion; and (3) the MCP subsystem can

maintain connections to multiple MCP servers (0..n) over different transports, enabling independent remote tool invocations. These forms of concurrency serve responsiveness (streaming/UX) and extensibility (multiple tool backends), while preserving a centralized orchestration model in packages/core.

Concurrent but asynchronous (non-blocking, event-driven I/O):
A major source of concurrency is asynchronous waiting on external dependencies. At the architecture level, core must communicate with the Gemini API over the network; this naturally introduces an asynchronous boundary where the request is in-flight while the system can manage UI streaming and internal state transitions. Similarly, MCP integration is explicitly described as connecting to configured servers via transports such as stdio, SSE, or HTTP, fetching tool definitions, and later executing tool calls, each of which is an I/O-driven interaction that is well-modeled as asynchronous activity. Importantly, this does not mean core is "making decisions in parallel"; rather, the runtime is structured to remain responsive and to handle multiple outstanding I/O interactions (especially across multiple MCP servers) without freezing the CLI experience.

Concurrent but synchronous (blocking waits on completion / hard gates):
Some concurrent activities are implemented using separate processes or isolated environments, but are synchronous from the orchestrator's perspective because core typically blocks the agent loop until results are returned. Tool execution is the clearest example: once core invokes a tool (built-in tools or MCP tools), the agent loop conceptually waits for that tool's result before proceeding to the next model step, because the tool output becomes required input to the subsequent prompt. The same applies to sandboxing: sandboxing introduces an isolation boundary for risky operations (shell commands, file modifications), but conceptually it functions as a controlled execution context for tools. The agent still pauses until the sandboxed tool finishes and returns outputs, and the user-approval gate remains a strict synchronization point before execution begins. In other words, the system uses concurrency to run work elsewhere (e.g., a sandbox/container or subprocess), yet the architectural control flow remains synchronous at each tool boundary because core serializes "action then observe result."

What are the implications for division of responsibilities among participating developers?

The split between packages/cli and packages/core creates a clean division of concerns that shapes how development work is organized. At a high level, Gemini CLI follows a centrally orchestrated model: the core runtime handles communication with the language model and manages tool execution, while the CLI layer takes care of everything the user sees and interacts with in the terminal. This setup naturally supports parallel development across subsystems, as long as the shared interfaces and event flows between them remain stable.

Interface Responsibilities (packages/cli):

The packages/cli subsystem owns the terminal-facing side of the application. Developers working on this layer focus on command parsing, interactive rendering, session-level UI state, and presenting streamed model responses. In practice, this means handling slash commands, showing confirmation prompts before tool execution, and managing configuration or settings dialogs.

Although the CLI layer does not implement model reasoning, it is far from isolated from the system's internal behavior. Interface contributors need to understand how streaming events, tool invocation requests, and approval signals flow out of the core. Handling these events correctly is essential for keeping the user experience responsive and consistent. This means that CLI and core contributors need to coordinate closely whenever changes are made to event structures or confirmation workflows.

Core Runtime Responsibilities (packages/core):

The packages/core subsystem houses the orchestration logic that drives interaction with the Gemini model and manages tool execution. Contributors working in this area handle prompt and context assembly, the model streaming loop, and session state tracking. They also maintain the tool registry, schedule tool execution, evaluate policies, and manage sandbox and input/output boundaries. Logging and reliability mechanisms fall under this layer as well.

It is worth noting that core developers do not control the Gemini model's internal algorithms. Rather, they define how the system interacts with the model—how requests are structured, how responses are interpreted, when tools are invoked, and how tool outputs feed back into subsequent prompts. Because this subsystem sits at the center of model interaction and tool execution, changes here can ripple across the entire system. Careful review and a commitment to interface stability are therefore essential when evolving the core runtime.

Tool and Integration Responsibilities:

Gemini CLI's architecture also draws a clear line around who is responsible for extending functionality through tools.

Developers of built-in tools implement specific capabilities and define structured input and output schemas. Their job is to ensure predictable behavior and safe interaction with system resources. That said, execution control and policy enforcement still live in the core subsystem.

The Model Context Protocol (MCP) takes this a step further by allowing external tools to be implemented outside the main repository entirely. MCP servers communicate with the CLI through supported transports, including standard I/O (stdio), Server-Sent Events (SSE), and HTTP-based streaming. This means independent teams can build

new integrations without touching the central orchestration logic, so long as they adhere to the MCP contract. In turn, core maintainers are responsible for keeping the MCP interface compatible rather than maintaining each external integration directly.

Security and Governance Responsibilities:

Tool execution is gated by a policy and confirmation mechanism that introduces an important boundary in the control flow. Before the system performs any action that could affect the local filesystem, shell environment, or network, it may require user approval, depending on how the configuration and policies are set up.

Work in this area involves defining default policy configurations, managing administrative and user-level overrides, and reviewing tool behaviors that touch sensitive resources. Special attention is needed for non-interactive or automated environments, where the system must avoid blocking indefinitely while waiting for user input. Ensuring safe behavior under these conditions is a key part of the overall architectural responsibility.

Concurrency and Coordination Considerations:

While model interaction follows a turn-based pattern, the system as a whole operates in an environment with significant asynchronous input and output. Communication with the Gemini API, interactions with MCP servers, and subprocess-based tool execution all require careful handling to avoid blocking the main interaction loop.

As a result, contributors to the core subsystem need to be mindful of non-blocking design practices. A blocking operation in the orchestration layer could degrade overall responsiveness. This makes code review and coordination especially important when introducing changes that touch model streaming or tool execution pathways.

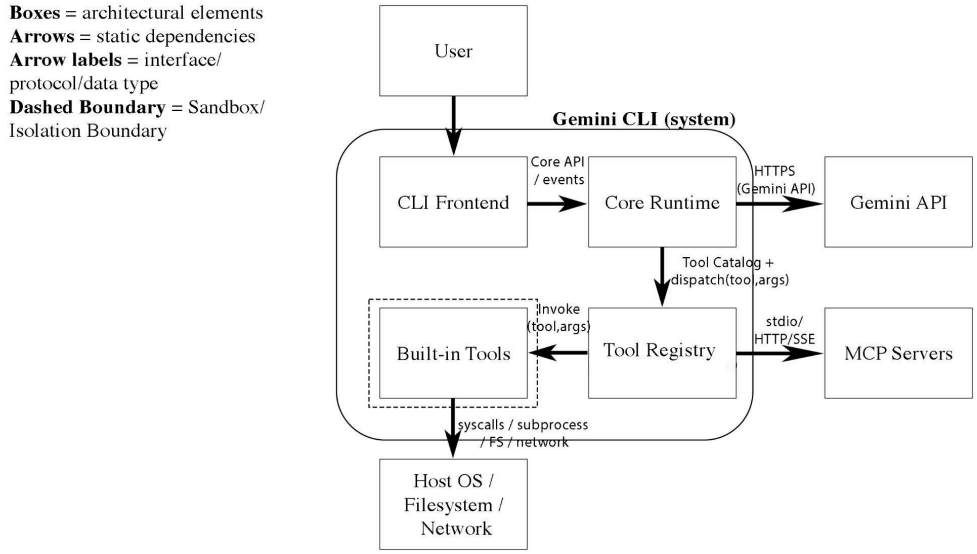
Implications:

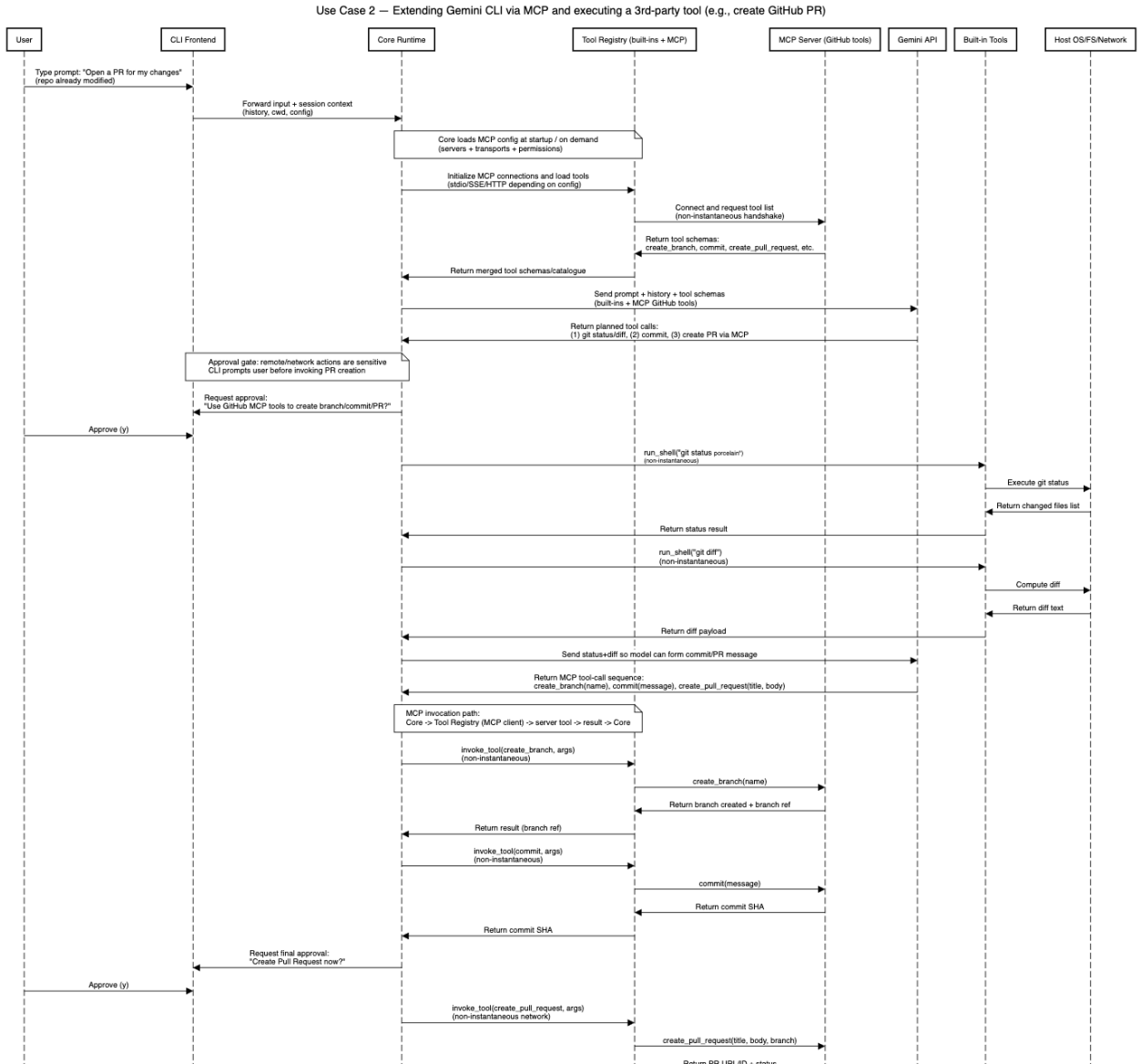
Taken together, the division of responsibilities in Gemini CLI reflects a platform-oriented architecture built around coordinated orchestration and controlled extensibility. The CLI layer owns user interaction, the core layer manages model communication and execution flow, tool developers extend the system's capabilities, and governance mechanisms regulate sensitive operations. These boundaries give different parts of the system room to evolve independently while still maintaining predictable behavior overall.

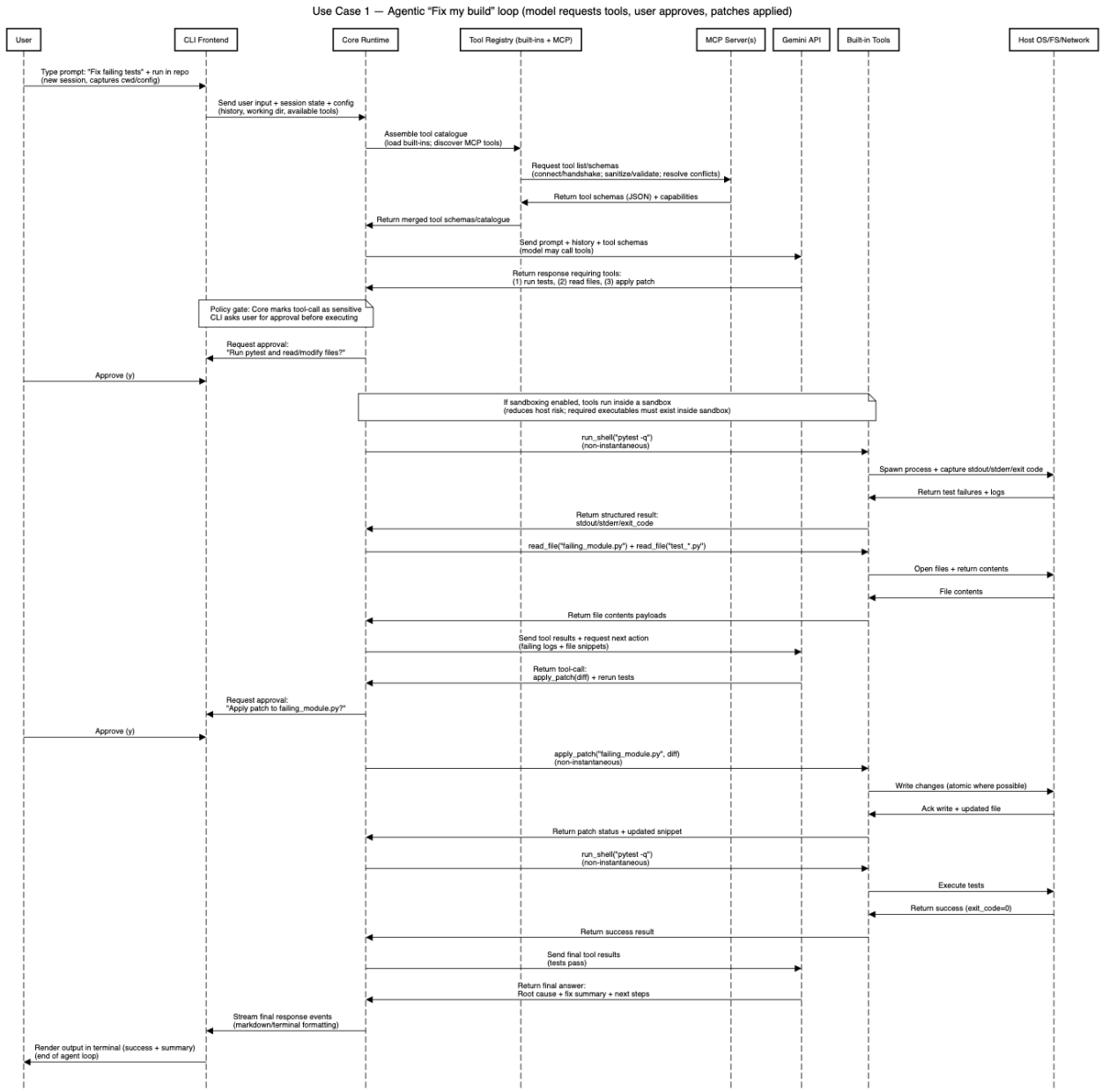
At the same time, this architecture depends on stable interfaces between subsystems and thoughtful coordination whenever those interfaces change. The separation of concerns supports modular development, but it also demands discipline, maintaining compatibility, reviewing policy changes, and preserving the system's interaction structure as it continues to evolve.

Diagrams:

Figure 1: Gemini CLI High Level System Architecture and Component Interactions







External Interfaces

Gemini CLI communicates with eight external interfaces: the Gemini API, the file system, the shell, web resources, MCP servers, configuration files, extensions, and the terminal.

The Gemini API interface exchanges prompts and responses bidirectionally over HTTP/2. Outbound transmissions include user prompts, conversation history with full context, tool definitions as FunctionDeclaration JSON schemas, system instructions from GEMINI.md files, model selection parameters, and generation settings like temperature and token limits. Inbound transmissions include streaming text chunks delivering partial responses incrementally, function call requests containing tool names

and JSON parameters, function call IDs for tracking, usage metadata with token counts, and finish reasons indicating completion status.

The file system interface handles bidirectional file operations. Outbound writes include file creations, modifications, configuration, updates to settings.json, session persistence and GEMINI.md memory files. Inbound reads include source code content, directory listings, pattern-based file discovery via glob, text searches via grep, hierarchical configuration files, and extension manifests.

The shell interface executes commands bidirectionally. Outbound transmissions send shell command strings including git operations, build commands, and test execution. Inbound responses return standard output, standard error, exit codes, and execution time.

The web interface primarily receives data inbound. HTTP/HTTPS requests fetch HTML pages, JSON API responses, documentation, GitHub content, and package registry data from npm and PyPI.

The MCP server interface operates bidirectionally across three transport options: Stdio subprocess communication, Server-Sent Events streaming, and HTTP/2 JSON-RPC. Outbound transmissions request tool discovery and send tool invocation calls with parameters and OAuth tokens. Inbound responses provide tool definitions with JSON schemas, execution results, server capability metadata, and resource listings.

The configuration interface reads settings from multiple sources in priority order: user settings files in JSON format, project specific settings, environment variables as strings, and command-line arguments. Higher priority sources override lower ones, enabling both personal customization and enterprise policy enforcement.

The extension interface reads inbound from filesystem packages. Extensions provide gemini-extension.json manifests defining metadata and dependencies, prompt template directories, MCP server configurations, custom command definitions, hook modules implementing lifecycle events, and agent skill definitions.

The terminal interface exchanges user interaction bidirectionally. Inbound from users includes text prompts with multi-line support, keyboard shortcuts for control, @-commands and slash commands for special functions. Outbound to users includes streaming response text, tool execution status messages, approval prompts displaying diffs in unified format, error messages, progress indicators, and syntax-highlighted code blocks using ANSI escape codes.

Use Cases:

The first use case demonstrates automated bug fixing: a developer tells Gemini CLI about a crash, and the system searches error messages across files using grep, reads the problematic file to analyze the code, identifies that the bug is a missing null check,

proposes a fix using optional chaining, shows the developer a diff for approval, applies the change after approval, and runs tests to verify the fix worked. This activates all five architectural layers as the terminal captures input, the CLI layer manages the streaming conversation, the core orchestrates six different tool calls across multiple ReAct loop iterations, the tool layer executes file reads and modifications with policy-based approval checks, and external services handle the API calls, file system operations, and shell execution. The second use case shows documentation generation: a developer requests comprehensive project docs, and the system discovers the project structure by listing directories and reading key files, analyzes fifteen Python modules to understand the codebase, generates README and API documentation following NumPy style conventions, creates persistent memory about the project for future conversations, and along the way uses conversation compression when approaching token limits and multiple specialized AI models for classification and summarization tasks, demonstrating the multi-model orchestration and performance optimization strategies.

Data Dictionary:

Naming Conventions:

- Stdio = Standard Input/Output (the text streams programs use)
    - stdin = where a program reads input from
    - stdout = where a program writes output to

Conclusions:

In the current report, the architecture of the Gemini CLI system was discussed, including its organization by structure, control and data flow, concurrency model, and adn architectural evolution. The review revealed that the system is designed with a centralizedd core backend, which coordinates with the user interface, language model services, and external tools. This client-server style design, with modular subsystems and well-defined extension points, promotes maintainability as well as long-term evolution.

The architecture focuses on a strictly predictable, sequential agent environment where the user input is handled by the core, possibly enhanced by the execution of some tools, and reentered into the user interface. Although concurrency exists by the means of asynchronous I/O, external processes and tool integrations, the general control flow is deliberately serially a to facilitate reliability, auditability and user control. The design would be responsive and safe at the same time, since approval gates and synchronisation points are placed around operations that may be potentially impactful.

In terms of architecture, the system exhibits a good separation of concerns. There are four layers of CLIs, core orchestration logic, tool subsystem and policy/safety layer that evolve relatively independently, allowing the addition of new features and integrations without destabilizing the system. Asynchronous boundaries and modular execution

environments like sandboxes and subprocesses isolate and optimize performance-critical paths, especially, model communication and tool execution.

Overall, the Gemini CLI architecture has a good ability to cope with the concepts of extensibility, controlled concurrency and incremental evolution. Its modular and stacked architecture enables the system to grow both in simplicity as a command-line helper and to more intricate agent-based processes without affecting stability and usability. This renders it a sound basis in terms of future expansion and incorporation of AI-assisted tools.

Lessons Learnt:

Analyzing Gemini CLI's architecture taught us to start with package structure before examining component details, as attempting to understand interactions without first mapping the monorepo boundaries caused initial confusion. We learned that production AI systems are fundamentally different from traditional applications: Gemini CLI uses 15+ specialized model configurations orchestrated together rather than being a simple API wrapper, a critical pattern we initially overlooked. Primary sources like official documentation and source code consistently proved more reliable than community blog posts, though secondary sources helped validate our interpretations. Maintaining the distinction between conceptual and concrete architecture required conscious effort, as we repeatedly caught ourselves documenting implementation specifics when the assignment required subsystem level descriptions. Finally, studying the evolution strategy proved as valuable as understanding current design: the public roadmap's explicit priorities explained why the architecture emphasizes frontend backend separation and extensible tool systems. For future analyses, we will establish boundaries first, recognize domain specific patterns, prioritize authoritative sources, maintain proper abstraction levels, and always examine how systems are designed to evolve.

AI Collaboration Report:

Model Selection - GPT-5.2, December 2025:
For this deliverable, our primary need was conceptual architecture analysis and technical writing, not large-scale code generation. That meant the AI teammate had to be strong at navigating a large codebase for architectural evidence and synthesizing it into accurate, clear explanations efficiently.

Out of the models the prompt engineer (Mack Rabeau) had access to, we selected GPT-5.2 as the best fit based on architecture comprehension, technical writing quality, ability to extract and cite concrete repository evidence and cost/latency suited to an analysis-heavy phase. We considered more code-specialized or higher-capacity options (e.g., GPT-5.3 Codex and Opus 4.6), but deep implementation capability was not the bottleneck at this stage and would have increased resource usage without proportional benefit. GPT-5.2 provided the best balance; it could search the codebase quickly, connect evidence across modules into a coherent conceptual model, and produce technical prose, without the heavier footprint of models optimized for extensive coding.

Tasks Assigned to AI Teammate:

The prompts used in this assignment can be divided into the following three categories :

- Codebase discovery: “Find X in the repo and show how it enables Y”

Task: Locate where specific architectural topics live in the code/docs (major subsystems, boundaries, control points), and return the relevant modules so we could cite evidence.
Why AI: This is high volume search work where AI speeds up locating the right places to read.

- Draft verification against the repository (proofreading + correctness checks)

Task: Review our draft text and either rewrite for accuracy or flag questionable claims; in both cases, point to the repo areas that confirm/contradict the wording.
Why AI: Helps spot mismatches between high-level descriptions and what the code/docs actually implement, which we then confirmed by reading the referenced sources.

- Academic writing support (rephrasing and consistency)

Task: Rephrase paragraphs into a consistent academic/report style; tighten language; standardize terminology across sections (components, flows, responsibilities).
Why AI: Improves clarity and cohesion efficiently, while humans retain responsibility for technical correctness and citations.

Interaction Protocol and Prompting Strategy:
Interaction with the AI was coordinated through a single designated prompt engineer (Mack Rabeau). Mack handled most prompts directly, then shared the AI’s outputs with the group for discussion. Other members contributed by proposing questions to investigate, reviewing whether the AI’s explanations matched our understanding of the architecture, and reviewing the AI’s academic writing for clarity, tone, and consistency with the report.

Our workflow was iterative and evidence driven. We typically started with a broad prompt to identify relevant subsystems, then followed up with narrower prompts that required traceability (specific files/modules and what role they played). Once we had the relevant sources, we drafted our own initial text based on the findings we had reviewed. We then used the model primarily as an accuracy check and revision aid: we asked it to critique our draft, flag any mismatches with the repository, and point to the specific areas in the docs/code that supported or contradicted our claims. This let us iteratively refine sections while keeping validation tied to primary sources.

Validation and Quality Control Procedures:
As mentioned previously, we treated the AI primarily as an accuracy checker and revision aid, and kept validation tied to primary sources. Concretely, our quality-control workflow was:

- **Source anchoring:** any AI-assisted claim had to map to at least one concrete source before it stayed in the report.
- **Technical section verification:** for each technical report section, we used the AI to critique our draft, then we cross-checked the relevant docs/code ourselves, and

finally used the AI's feedback to tighten the writing, if needed, without changing the underlying meaning.
- **Revision loop:** we repeated this cycle until the section was correct, clear, and consistent with the rest of the report.

Quantitative Contribution to Final Deliverable
We estimate the AI contributed ~40% of the final submitted deliverable. This contribution was mainly in speeding up research and improving clarity, while humans remained responsible for correctness and final decisions (codebase/documentation discovery ~25%, draft critique for accuracy: ~10%, academic writing support: ~5%)

Reflection on Human–AI Team Dynamics
Using an AI teammate saved time during early exploration and drafting, but also created additional work in prompt iteration and validation. It improved brainstorming by quickly surfacing plausible architectural "places to look", but we treated these as leads and made final decisions as a group based on what we could verify in the docs and code.

The main challenge was avoiding unsupported statements. We handled this by requiring file/document pointers and iterating until sections were correct. The biggest lesson we will apply in future projects is to use AI for search, synthesis and writing support, while enforcing a strict human verification gate (primary sources + team review) before anything enters the final report.

References:

Official Gemini CLI Architecture Documentation https://geminicli.com/docs/architecture/ Official architectural overview describing system design, component structure, and design patterns

Gemini CLI Architecture (GitHub)
https://github.com/google-gemini/gemini-cli/blob/main/docs/architecture.md Detailed architecture documentation in the source repository explaining package structure and core components

Gemini CLI Public Roadmap https://google-gemini.github.io/gemini-cli/ROADMAP.html Development priorities, feature areas, and evolution strategy with explicit prioritization principles

MCP Server Integration Documentation
https://google-gemini.github.io/gemini-cli/docs/tools/mcp-server.html Official documentation on integrating Model Context Protocol servers for external tool discovery

Gemini CLI npm Package https://www.npmjs.com/package/@google/gemini-cli Official npm package page with installation instructions, version history, and basic usage

"Gemini CLI: Agentic Software Engineering is Here" by C. Barkin Ozer
https://cbarkinozer.medium.com/gemini-cli-agentic-software-engineering-is-here-f280c30e

862f Community analysis discussing agentic capabilities and real-world software engineering applications

"Mastering the Gemini CLI" by Creative AI Ninja https://medium.com/@creativeaininja/mastering-the-gemini-cli-cb6f1cb7d6eb Tutorial and guide covering advanced usage patterns and workflow optimization

Google Gemini CLI Cheatsheet by Philipp Schmid https://www.philschmid.de/gemini-cli-cheatsheet Quick reference guide with common commands, configuration options, and usage examples

DeepWiki: Gemini CLI Documentation https://deepwiki.com/waywardgeek/gemini-cli Community-maintained documentation with additional examples and architectural insights

Gemini CLI Technical Comparison Discussion https://gist.github.com/steipete/670b2522b476f6a850e07b3535ebb2b7 Technical discussion comparing Gemini CLI with other AI coding assistants and architectural approaches

"An Introduction to Software Architecture" by Garlan & Shaw (1994) https://www.cs.cmu.edu/afs/cs/project/able/ftp/intro_softarch/intro_softarch.pdf Foundational paper defining architectural styles (layered, pipe-filter, event-driven, etc.) referenced in this analysis

"SAAM: A Method for Analyzing the Properties of Software Architectures" by Kazman et al. (1994) https://citeseerx.ist.psu.edu/viewdoc/download?doi=10.1.1.65.8786&rep=rep1&type=pdf SEI architectural analysis method required for Assignment 3

Model Context Protocol (MCP) Specification https://modelcontextprotocol.io/ Standard protocol for AI-tool integration that Gemini CLI implements for extensibility

Ink - React for Terminal Interfaces https://github.com/vadimdemedes/ink React renderer for terminals used by Gemini CLI's frontend for component-based UI

Google Gemini API Documentation https://ai.google.dev/docs Official documentation for the Gemini API that Gemini CLI wraps and orchestrates

Node.js Monorepos with npm Workspaces https://docs.npmjs.com/cli/v7/using-npm/workspaces Documentation on npm workspaces pattern used for Gemini CLI's monorepo structure